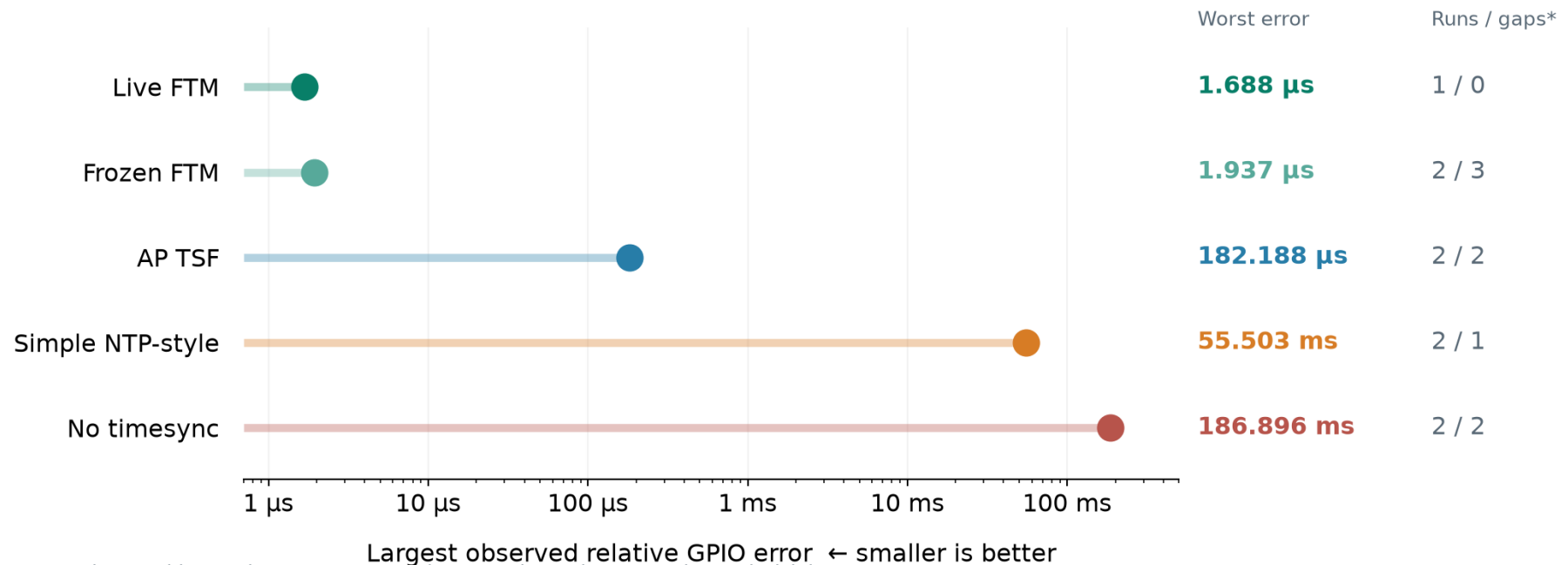


Best-supported choice: live FTM + MCPWM

Clock synchronization methods • measured relative to node 2

Which clock method aligned the pulses best?

Same firmware • MCPWM output • camera OFF • 60-second records



* Gaps = observed intervals >1.5 s, summed across channels. Log scale: each tick is 10×

Nine completed trials; finite maxima of delivered pulses, not guaranteed bounds. No-sync phase depends on boot epochs.

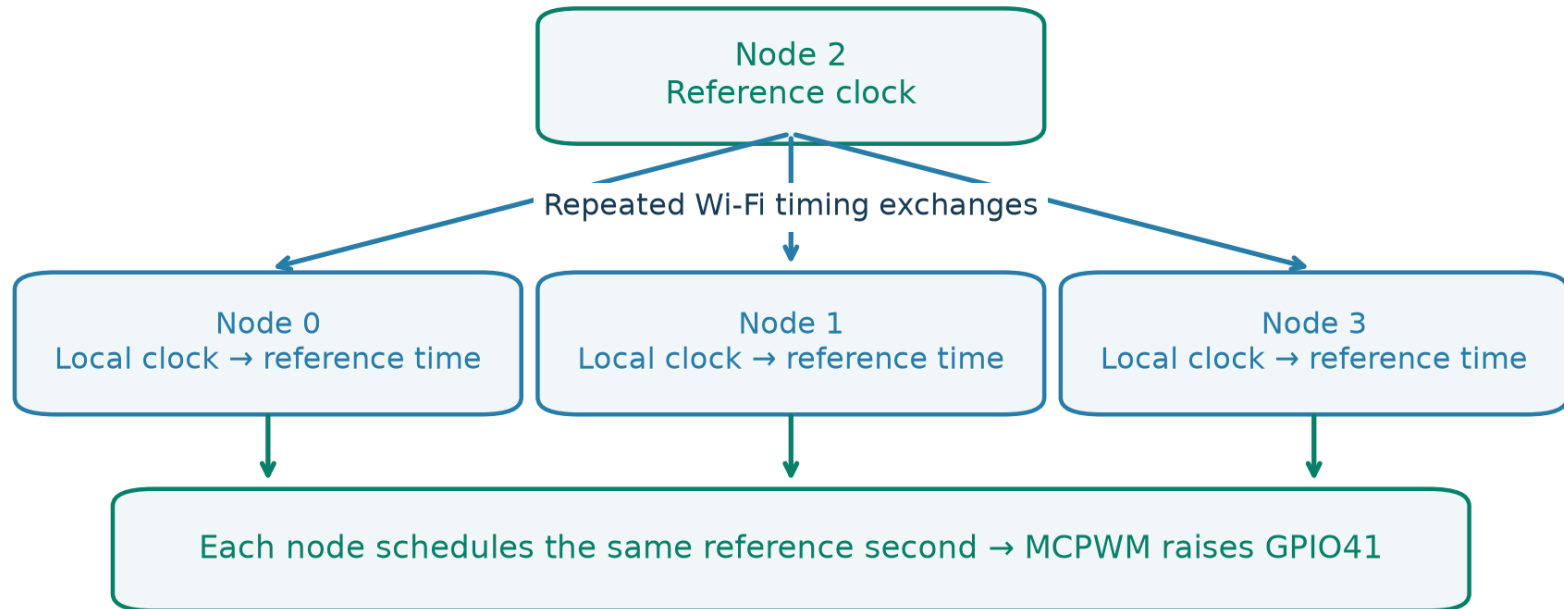
Why FTM? Its new trial had a largest observed error of 1.688 microseconds and no missed cycles. Three separate historical repaired trials stayed below 1.937 microseconds. Frozen FTM was similarly close for short periods, but had missing cycles and stops updating its clock model.

Limits: One new FTM trial versus two per other source; analyzer timeouts stopped the planned repeats. These are GPIO results, not camera-exposure accuracy. Simple NTP-style is our basic private UDP estimator, not a complete NTP client.

How FTM synchronizes the nodes

A clock can have the wrong starting value and run at a slightly different speed.

FTM builds a shared time coordinate



1. Measure. Nodes exchange Wi-Fi timing messages. FTM timestamps are captured near the radio events, reducing the uncertainty associated with application send/receive timing.

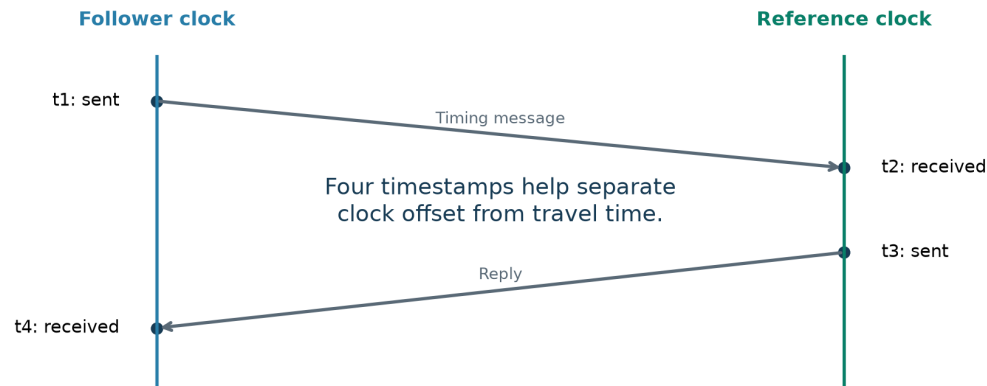
2. Learn the clock mapping. Repeated measurements estimate: $\text{reference time} = a \times \text{local time} + b$. Here a corrects clock speed and b corrects the starting offset. The local clock need not be physically reset.

3. Schedule. Convert the next common reference second into each node's timer deadline. MCPWM generates the GPIO edge. The analyzer checks how far apart the real edges arrived.

What changes between the five methods?

Same output mechanism; different ways of estimating a common time.

Two clocks exchange timestamped messages



Conceptual request/reply diagram, not an FTM packet trace. Downward = later time. Path asymmetry still causes error.

Software NTP-style: timestamps before send and after receive. This trial uses only the latest offset, with no frequency correction or delay filter.

FTM: radio-event timestamps feed a repeatedly updated offset-and-speed model. Timestamp units alone do not guarantee accuracy.

No timesync: use each local clock as-is. Different starting values produce arbitrary phase differences.

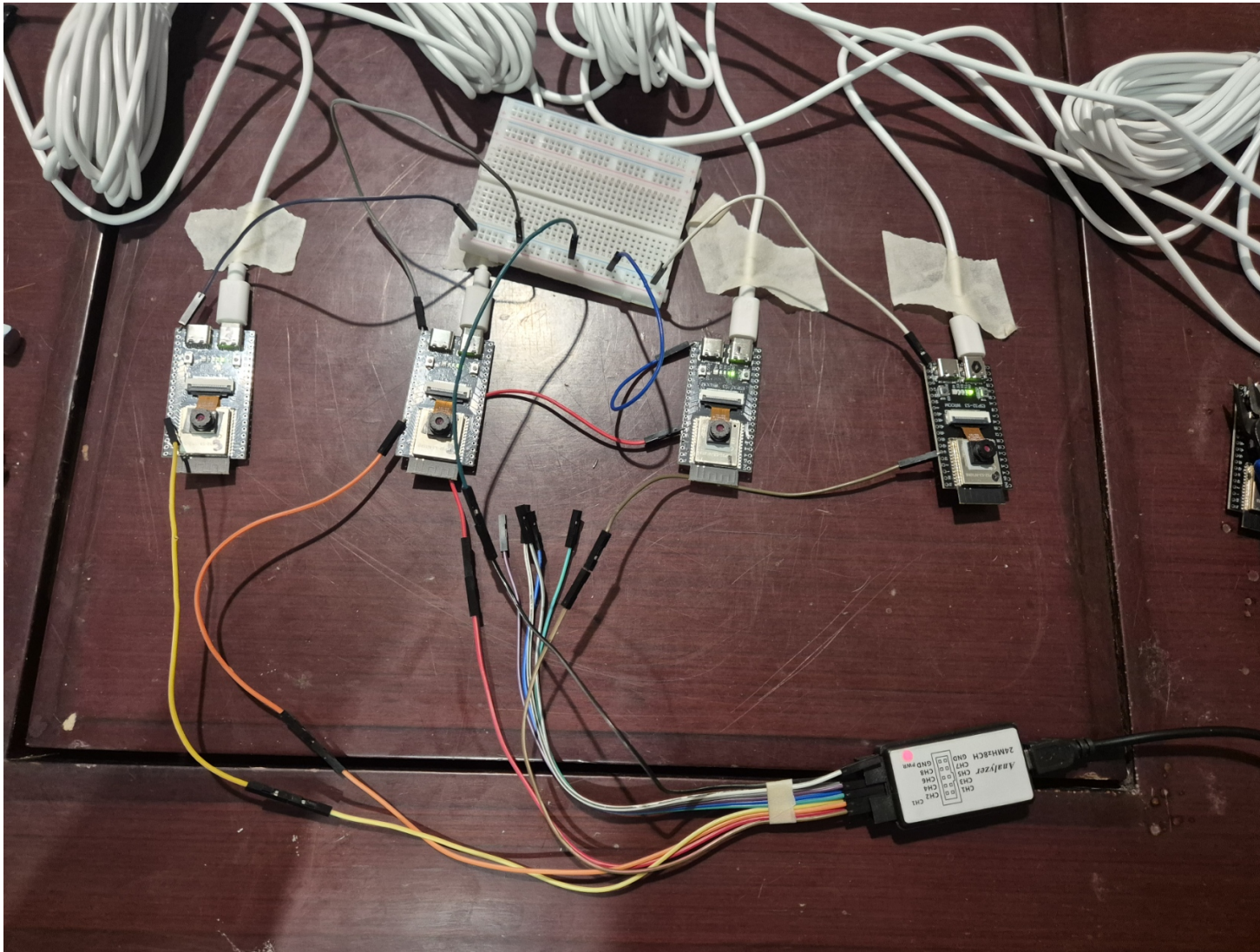
Frozen FTM: synchronize once, then retain the mapping. This tests holdover, not an unsynchronized clock.

AP TSF: read the hotspot's shared Wi-Fi timer. This uses the access-point clock rather than the node-2 MAC clock.

For the simple NTP-style exchange: $\text{offset} = [(t2 - t1) + (t3 - t4)] / 2$. The calculation assumes roughly symmetric travel times. Unequal paths and scheduling delay can bias it. Sources: RFC 5905 §8; archived implementation and full report.

The physical setup

Your supplied photograph • retained without image alteration



Four ESP32-S3 camera boards
USB provides power/connectivity.

Wi-Fi carries time information
The probe wires measure GPIO
outputs; they do not synchronize the
clocks.

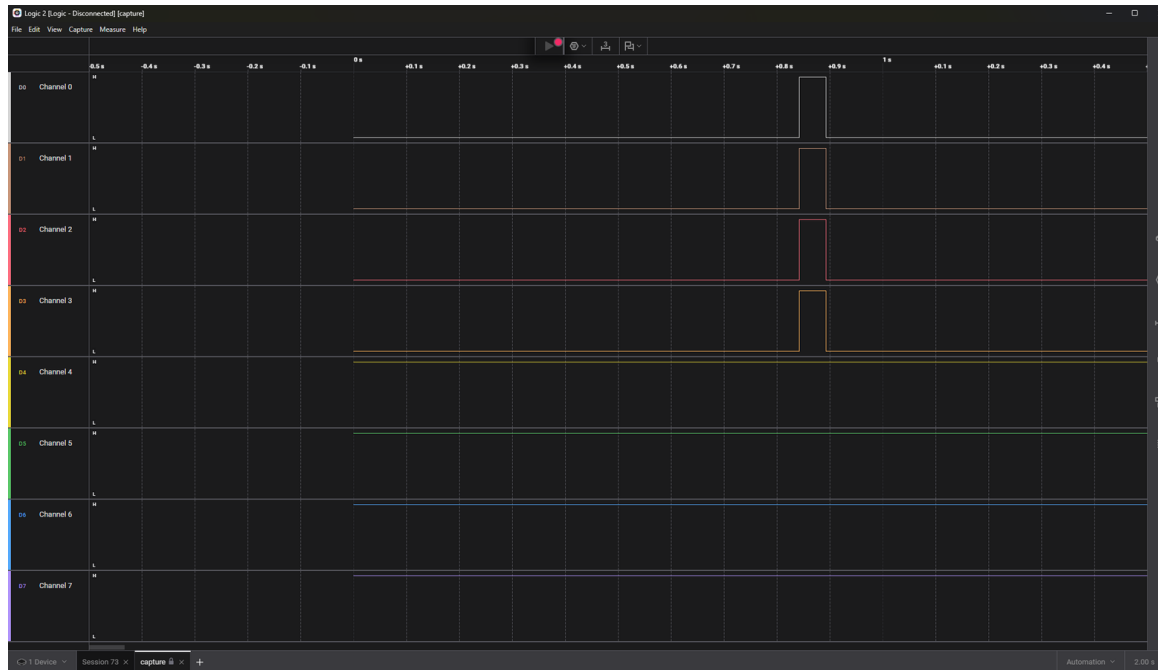
Four analyzer inputs
D0-D3 correspond to nodes 0-3 in the
experiment, with node 2 as reference
and a common ground.

Identification correction
The photo shows a USB analyzer marked
24 MHz / 8CH. “Logic” is its
software-reported identity, not verified
proof of manufacturer. Successful trials
used 16 MS/s.

Physical left-to-right board order and exact manufacturer/revision cannot be confirmed from this photo. The earlier report's “original Saleae Logic” wording should be read as a software device classification.

What the analyzer actually recorded

Archived live-FTM trial: 20260909-065110 / 00-00-mcpwm-ftm • no new acquisition

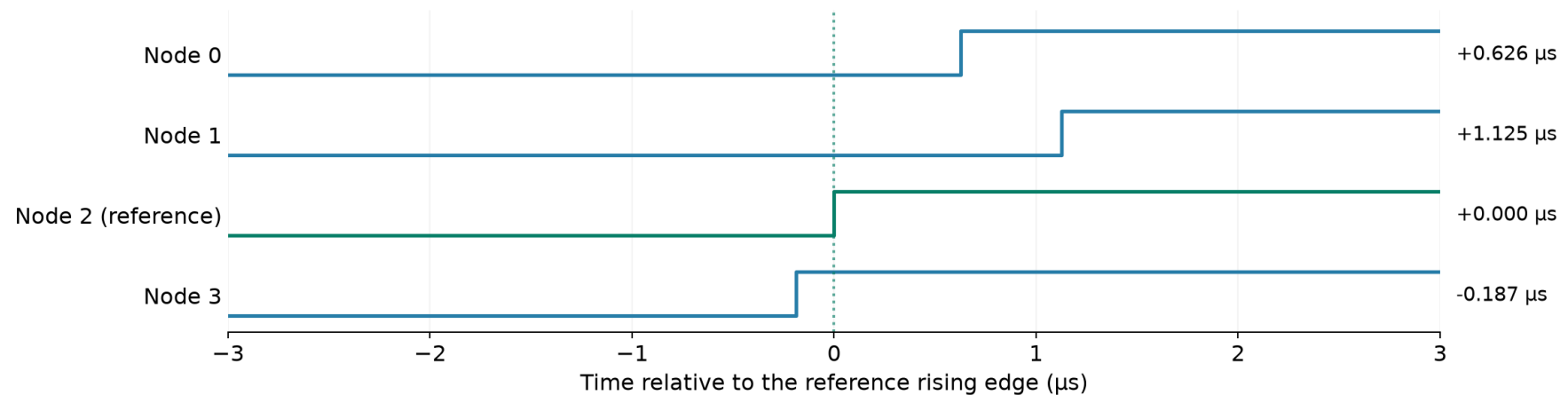


Actual Logic screenshot

D0-D3 show aligned 50-ms pulses. D4-D7 are unused.

At a two-second viewing scale, microsecond differences are invisible. The plot below magnifies one recorded rising edge.

One actual FTM event, magnified from the raw recording



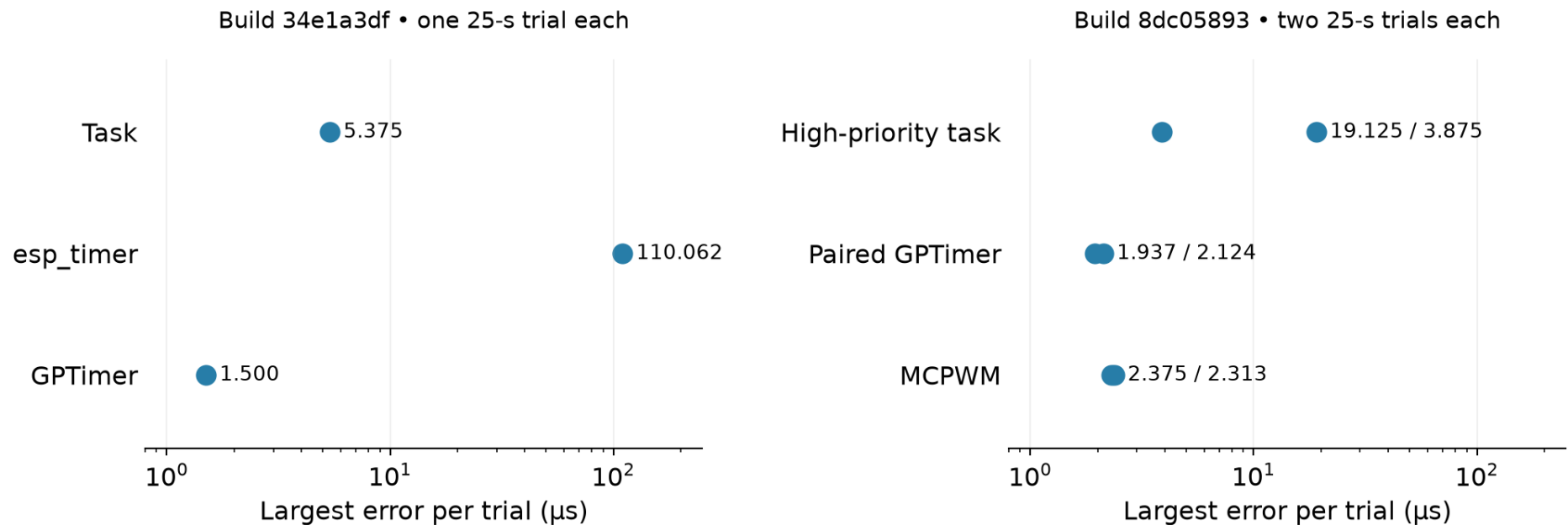
The lower plot is reconstructed from the same raw CSV, not a screenshot or simulated result. One event illustrates alignment; the comparison chart uses all qualifying matched pulses. Sampling interval: 62.5 ns, not calibrated accuracy.

The six output backends we also tested

Exploratory comparison • clock method held at FTM within each block

The timer used to generate the pulse matters too

All use FTM. Compare within each panel; firmware differs between panels.



Exploratory 25-second records, 25 pulses per follower. Later epoch failures and capture failures remain in the full report. These short pre-repair trials do not replace the longer repaired MCPWM validation.

Interpretation: GPTimer was the best of the first three short trials. In the later short block, paired GPTimer and MCPWM were close; high-priority task results varied. These panels use different firmware and cannot supply one fair six-way ranking.

Why recommend MCPWM overall? It is the backend with the retained clean, longer validation after the clock-epoch repair. Earlier short results, epoch failures and acquisition failures remain documented; they are not silently pooled into the five-source chart.

Data and definitions: ../timesync_report.pdf, ../analysis/run_metrics.json, figure_data.json. Mechanism sources: RFC 5905 §8; Espressif ESP-IDF v5.5.1 Wi-Fi FTM documentation; archived firmware. All diagrams are explanatory, not additional measurements.